- *Information retrieval*—Finally, we create an instance of a `Retrieval` object available from the LangChain package that takes user queries, converts them into embeddings, and searches against the stored document embeddings to find the most relevant matches. This process can involve techniques like searching for the nearest neighbor or other similarity measures.

Let's see how each of these steps works and how we can accomplish them using LangChain.

6.2.1 Document ingestion using text splitters

The goal of text splitting is to transform documents into smaller, manageable chunks. This process helps ensure that the text fits within the model's context window and maintains semantic coherence. LangChain provides various built-in document splitters that simplify this task, allowing you to manipulate documents effectively. The *text splitters* take input data and transform it into output data suitable for storage. LangChain offers several types of text splitters, as shown in table 6.2.

Table 6.2 LangChain text splitters

Name	Splits	Adds metadata?	Description/use case
Recursive	Specific character	Yes	Recursively splits text while keeping related pieces together; recommended for initial splitting
HTML	HTML characters	Yes	Splits text based on HTML tags and structures
Markdown	Markdown syntax	Yes	Splits text according to Markdown syntax
Code	Code-specific characters	Yes	Splits code based on syntax specific to various programming languages
Token	A token	No	Splits text on tokens, with various methods available for token measurement
Character	A single user-defined character	No	Simple splitting based on a specified character

All of these text splitters work by first splitting the text into small, semantically meaningful units (often sentences). These units are then combined into larger chunks until a specified size limit is reached. Once the size limit is hit, that chunk becomes a standalone piece of text and a new chunk begins, with some overlap to maintain context between chunks.

The `RecursiveCharacterTextSplitter` is LangChain's recommended splitter. It is a highly flexible tool that can be adapted to various types of documents and the specific requirements of your project. It can split text recursively based on a list of separators, starting with newlines and progressing to smaller separators. This approach helps preserve the semantic structure of the text as much as possible.

The following listing shows how to create and use a `RecursiveCharacterTextSplitter` instance to split a text document into a list of chunks ready for ingestion in the vector storage.